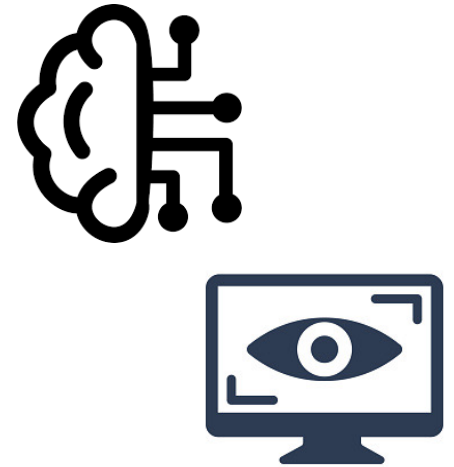


Deep Learning

Qingjie Meng

Assistant Professor
School of Computer Science
University of Birmingham



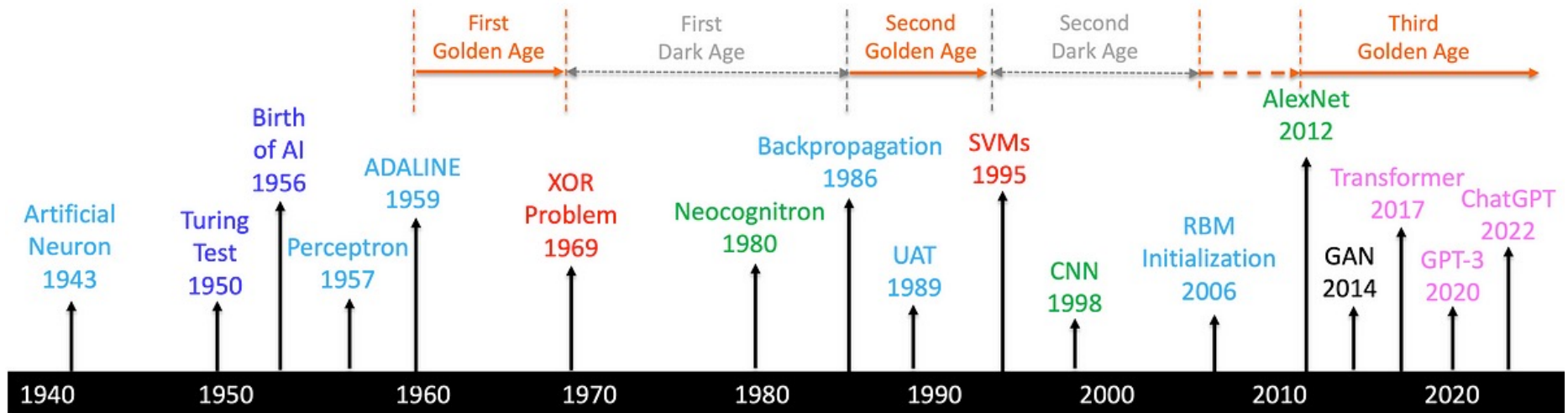
[34266][34256][LI Artificial Intelligence 2]

Rise of deep learning

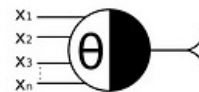
Interest over time ?



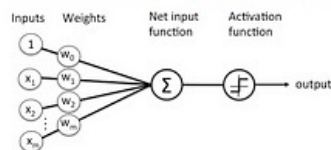
A Brief History of AI with Deep Learning



McCulloch-Pitts

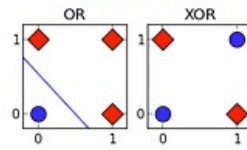


Rosenblatt

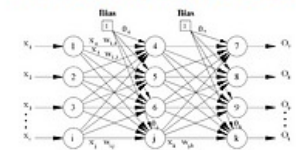


Widrow-Hoff

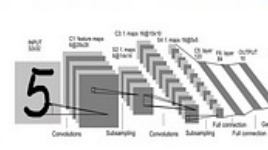
Minsky-Papert



Rumelhart, Hinton et al.



LeCun



Hinton-Ruslan Krizhevsky et al.



Vaswani



www.image-net.org

22K categories and **14M** images

- Animals
 - Bird
 - Fish
 - Mammal
 - Invertebrate
- Plants
 - Tree
 - Flower
- Food
- Materials
- Structures
 - Artifact
 - Tools
 - Appliances
 - Structures
- Person
 - Scenes
 - Indoor
 - Geological Formations
 - Sport Activities

Deng, Dong, Socher, Li, Li, & Fei-Fei, 2009

IMAGENET Large Scale Visual Recognition Challenge

The Image Classification Challenge:
1,000 object classes
1,431,167 images

Steel drum



Output:
Scale
T-shirt
Steel drum
Drumstick
Mud turtle



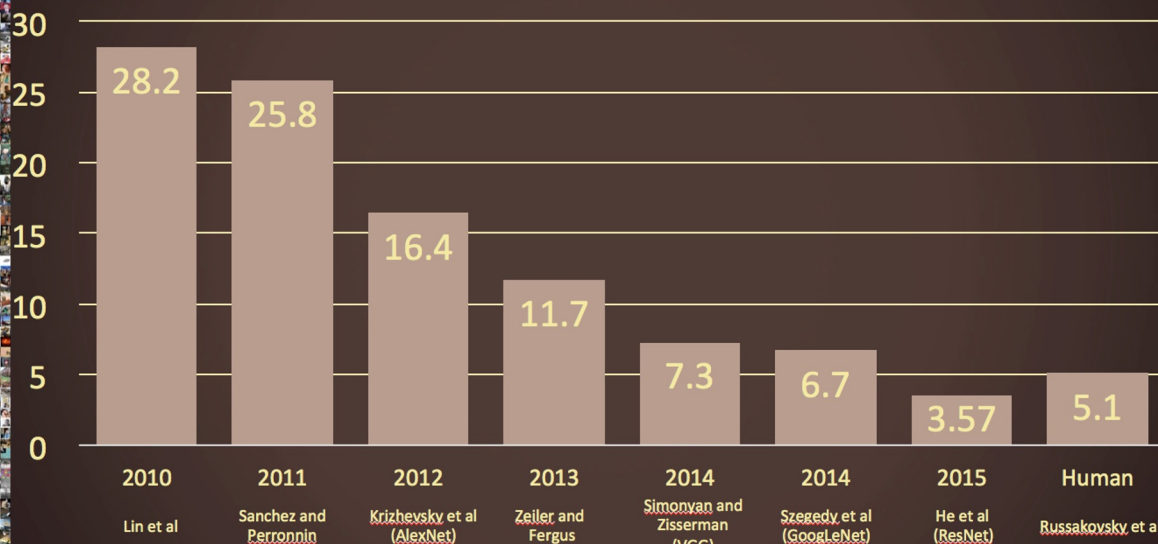
Output:
Scale
T-shirt
Giant panda
Drumstick
Mud turtle



Russakovsky et al. arXiv, 2014

IMAGENET Large Scale Visual Recognition Challenge

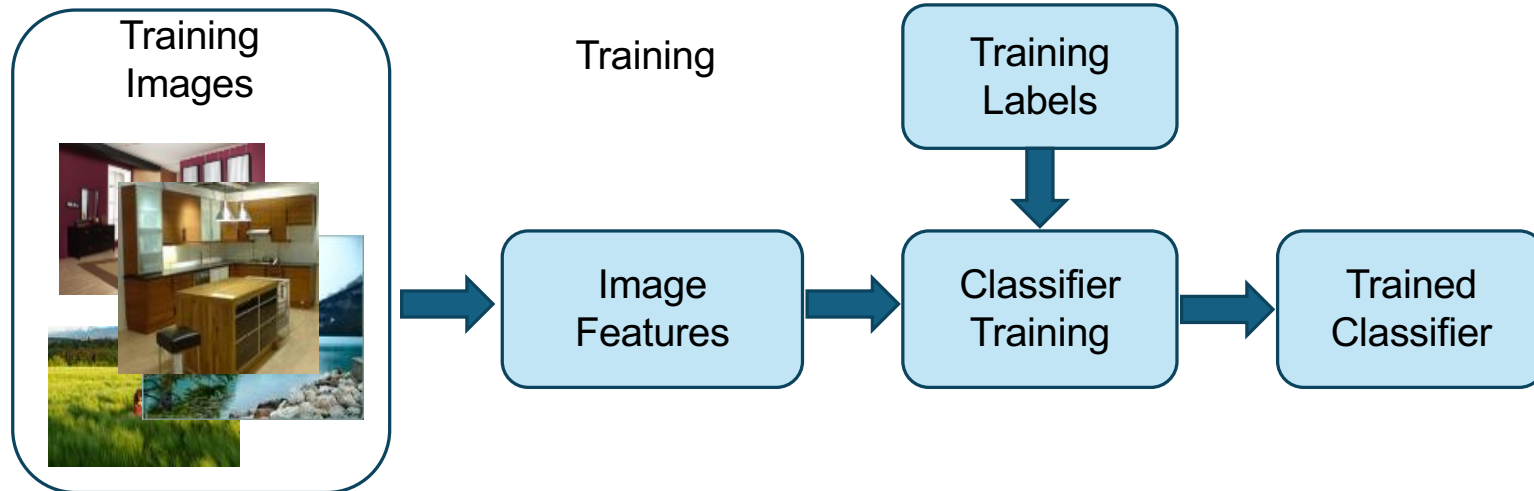
The Image Classification Challenge:
1,000 object classes
1,431,167 images



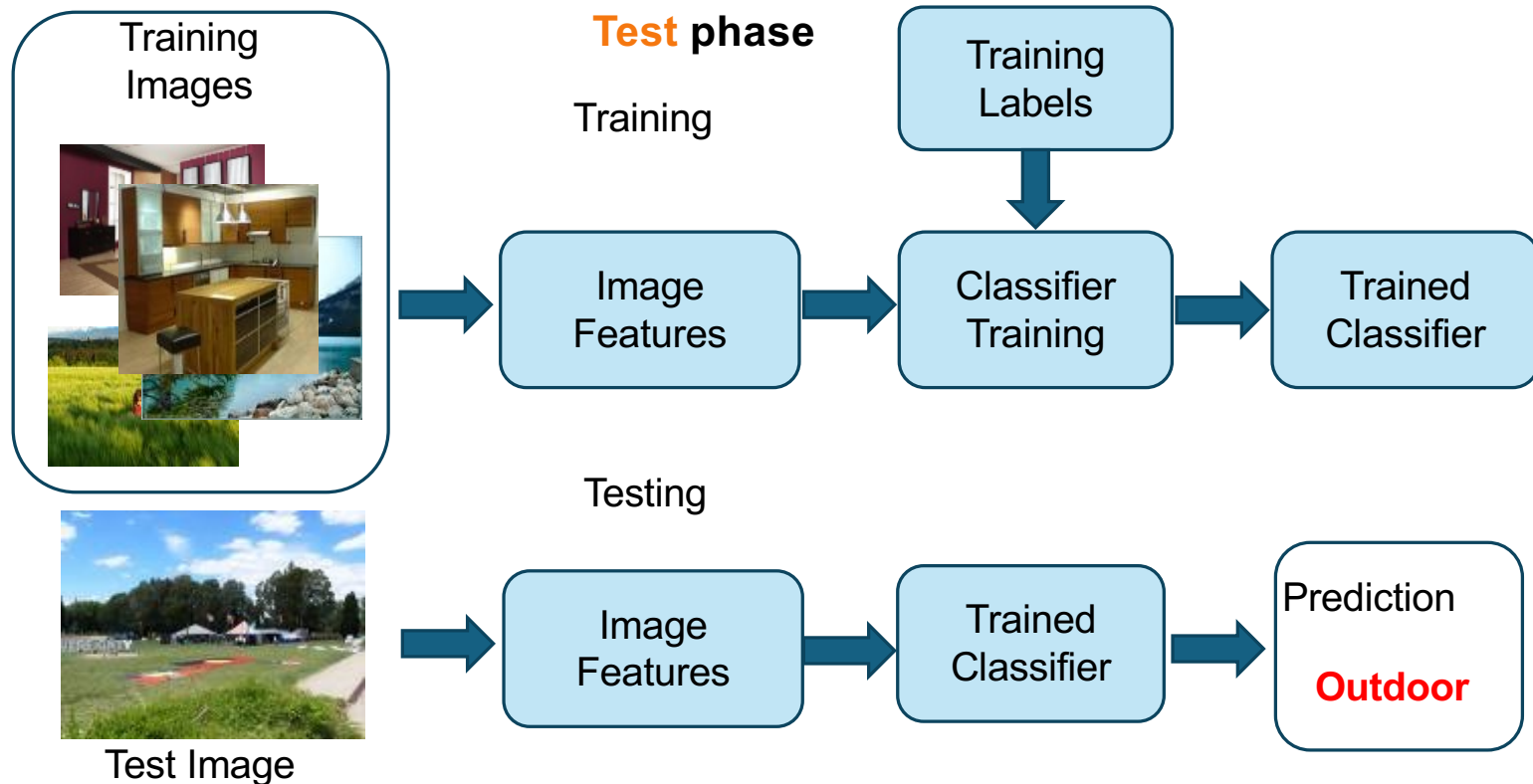
Russakovsky et al. arXiv, 2014

Traditional Image Categorization

Training phase



Traditional Image Categorization



Features have been key..

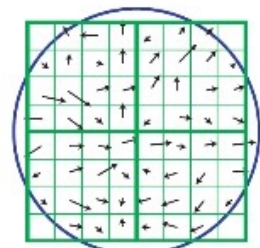
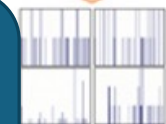
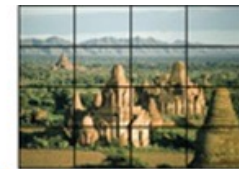
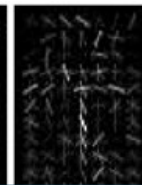
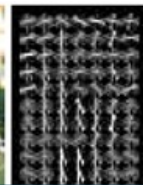
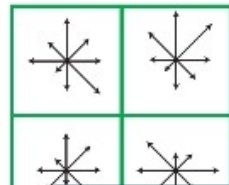
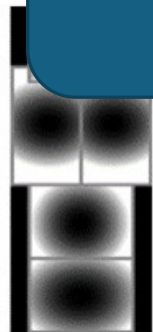
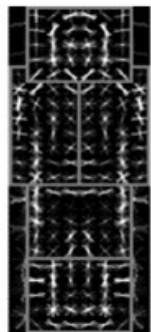
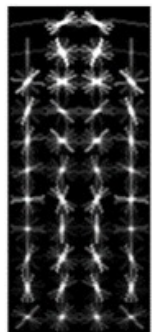


Image gradients

SIFT [Loewe IJCV



nik et al. CVPR 06]



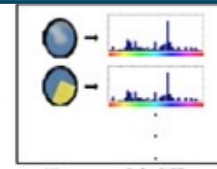
DPM [Felzenszwalb et al. PAMI 10]



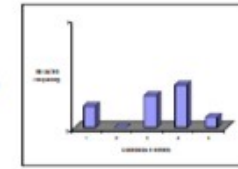
Image



Harris-Laplace detector



Histograms, ColorSIFT, ...



Vector quantization

Fixed-length feature vector

Color Descriptor [Van De Sande et al. PAMI 10]

Hand-crafted

What about **learning** the features?

- Learn a *feature hierarchy* all the way from pixels to classifier
- Each layer extracts features from the output of previous layer
- Layers have (nearly) the same structure
- Train all layers jointly (“**end-to-end**”)



Learning Feature Hierarchy

Goal: **Learn** useful higher-level features from images

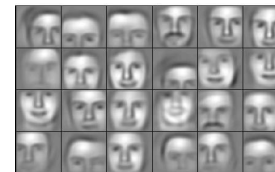
Input data



Lee et al., ICML 2009;
CACM 2011



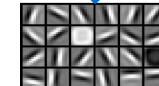
Feature representation



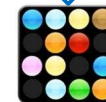
3rd layer
“Objects”



2nd layer
“Object parts”



1st layer
“Edges”

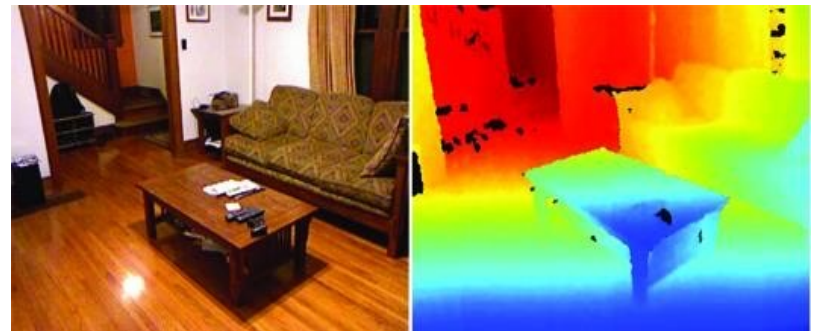


Pixels

Slide: Rob Fergus

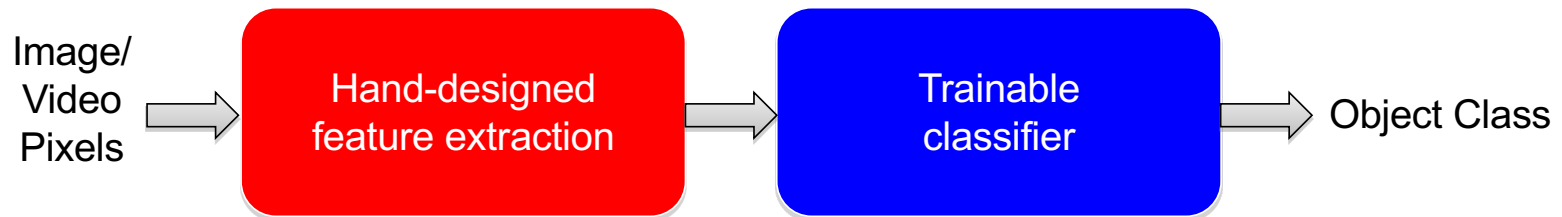
Learning Feature Hierarchy

- Better performance
- Other domains (unclear how to hand engineer):
 - Kinect
 - Video
 - Multi spectral
- Feature computation time
 - Dozens of features now regularly used
 - Getting prohibitive for large datasets (10's sec / image)



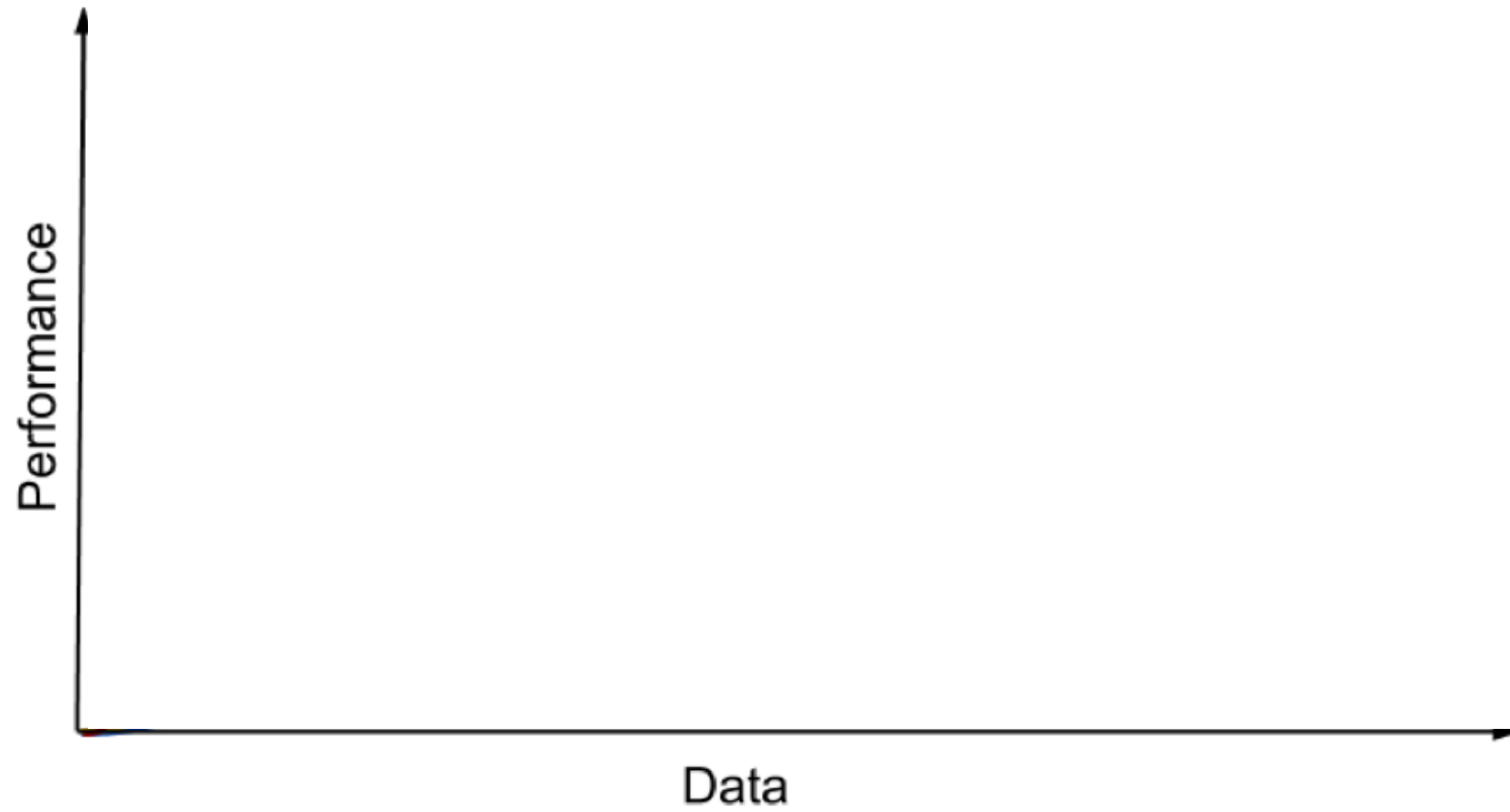
“Shallow” vs. “Deep” architectures

Traditional recognition: “Shallow” architecture

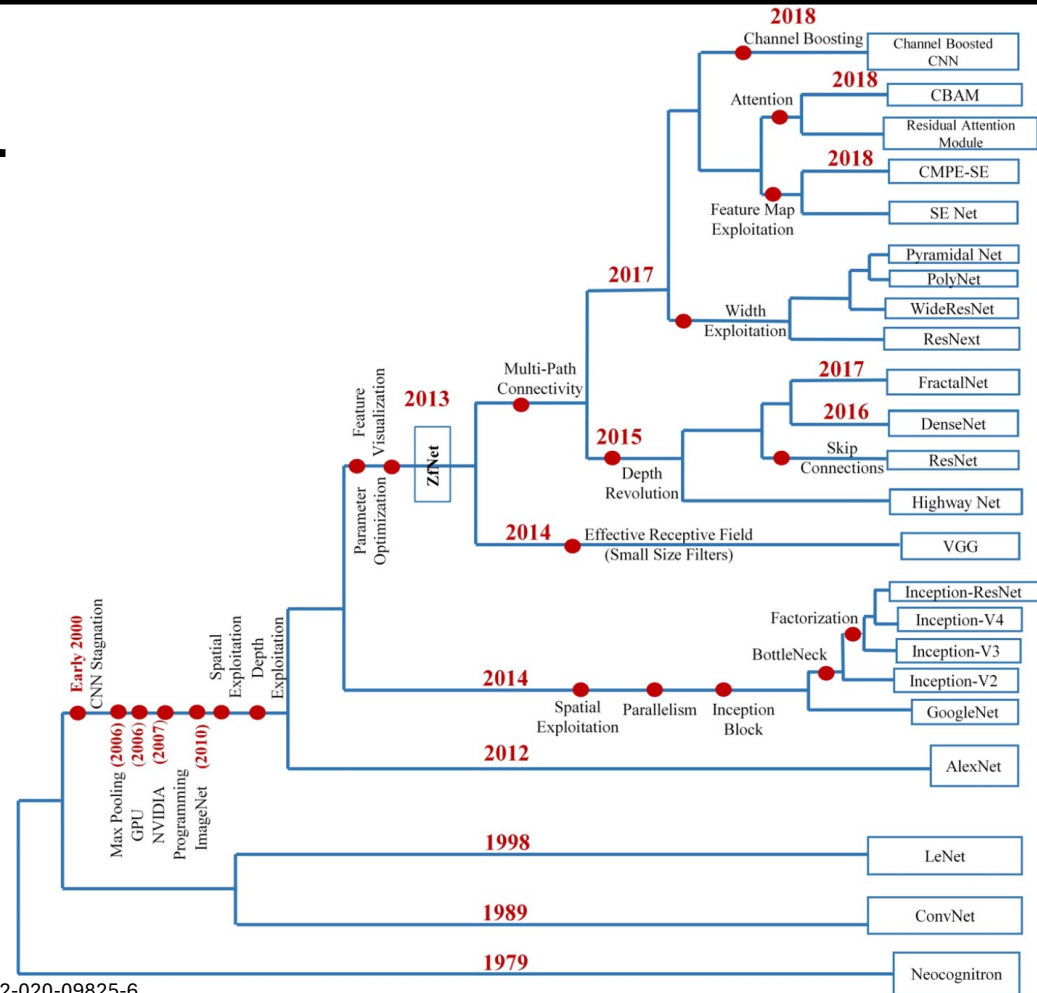


Deep learning: “Deep” architecture





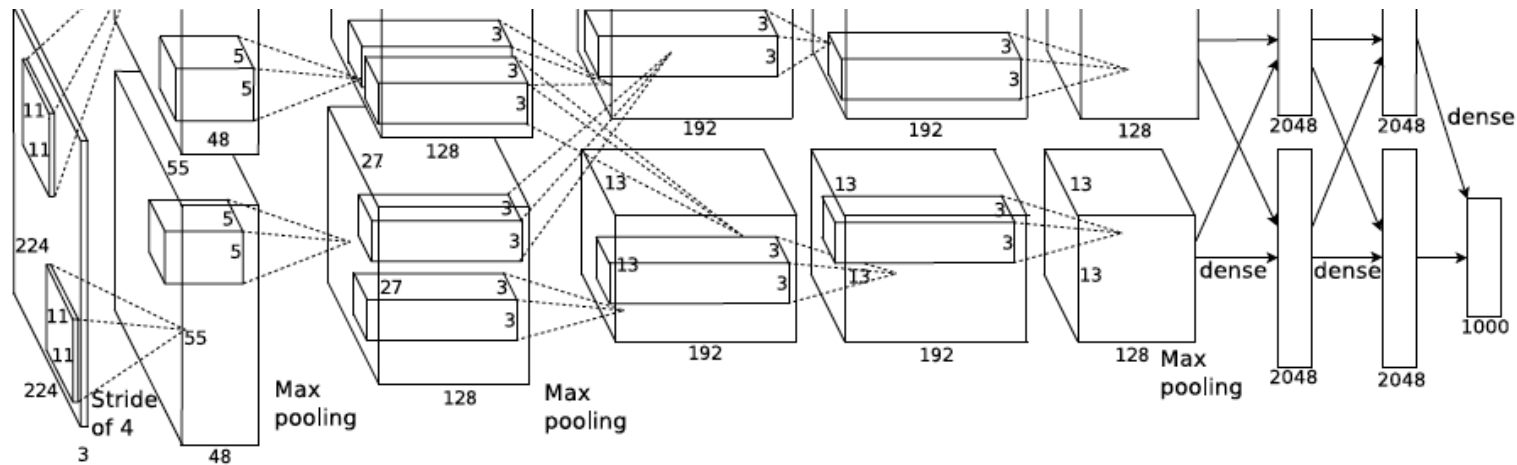
Classical



Classical deep neural networks

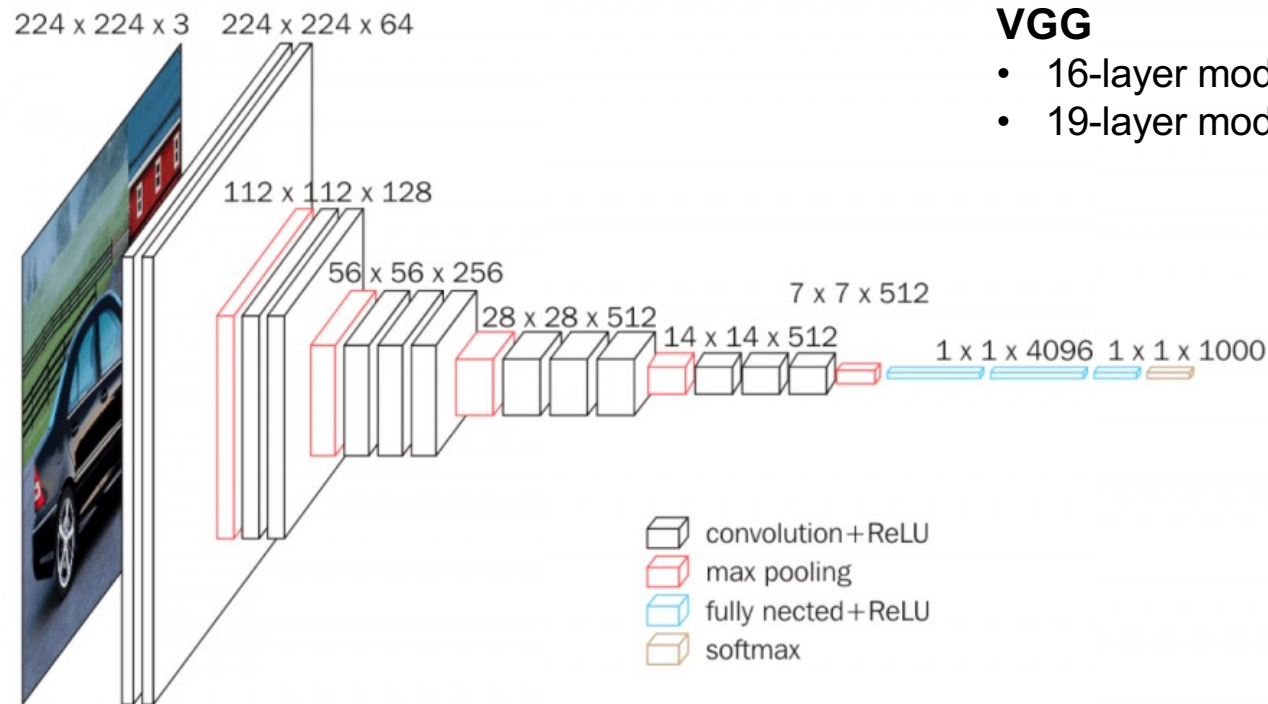
AlexNet

- 7 hidden layers, 650,000 units, 60,000,000 params
- More data (10^6 vs. 10^3 images)
- GPU implementation (50x speedup over CPU)
 - Trained on two GPUs for a week



A. Krizhevsky, I. Sutskever, and G. Hinton, [ImageNet Classification with Deep Convolutional Neural Networks](#), NIPS 2012

Classical deep neural networks



VGG

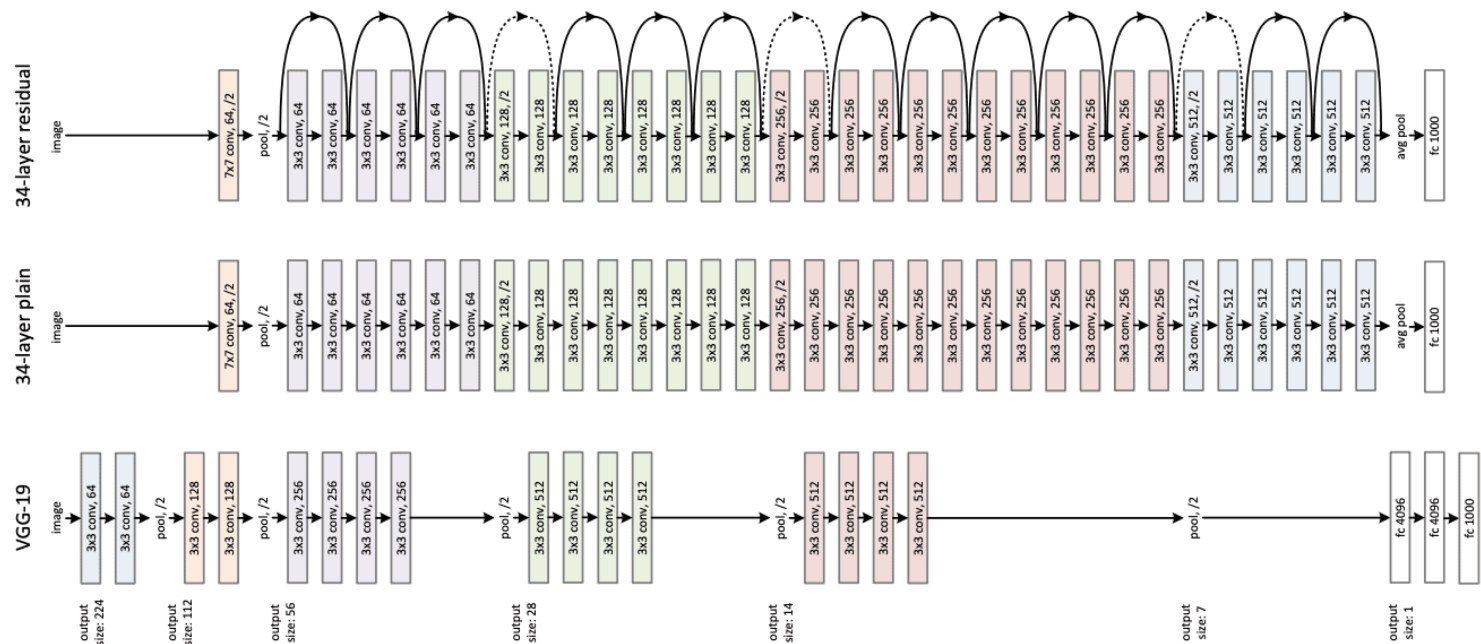
- 16-layer model, 528,000,000 parameter
- 19-layer model, 548,000,000 parameter

K. Simonyan, A. Zisserman, [Very Deep Convolutional Networks for Large-Scale Image Recognition](#), ICLR 2015

Classical deep neural networks

ResNet

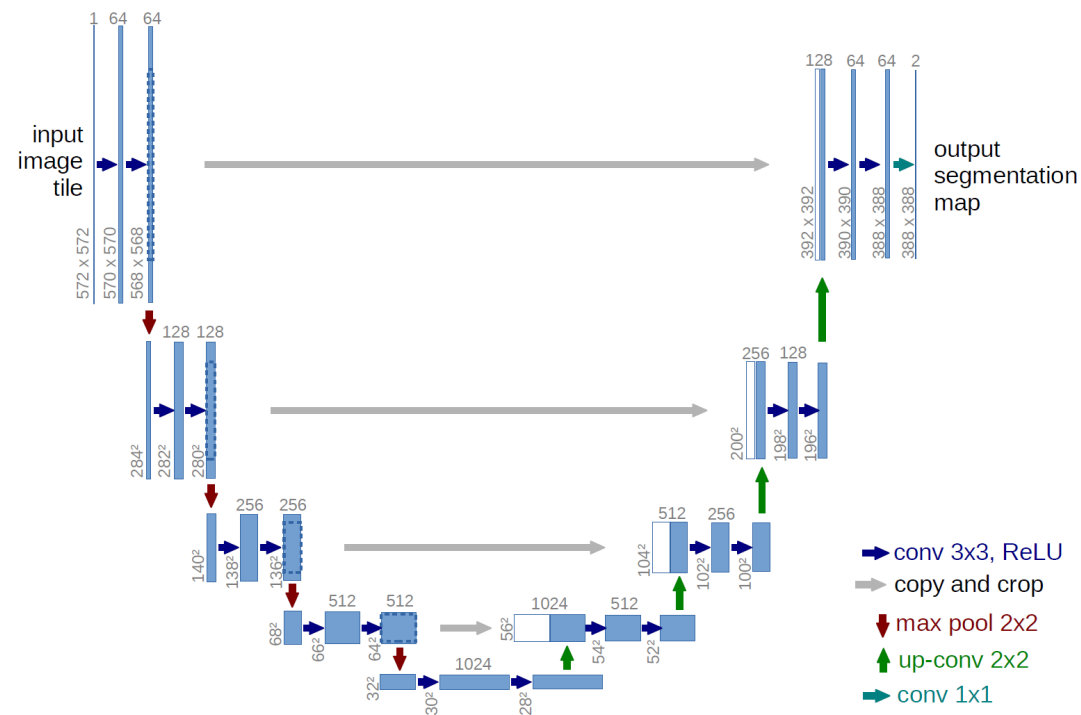
- ResNet-18
- ResNet-34
- ResNet-50
- ResNet-101



Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun, **Deep Residual Learning for Image Recognition**, CVPR 2016

Classical deep neural networks

U-Net



Olaf Ronneberger, Philipp Fischer, Thomas Brox, [U-Net: Convolutional Networks for Biomedical Image Segmentation](#), MICCAI 2015

Classical deep neural networks

Transformer

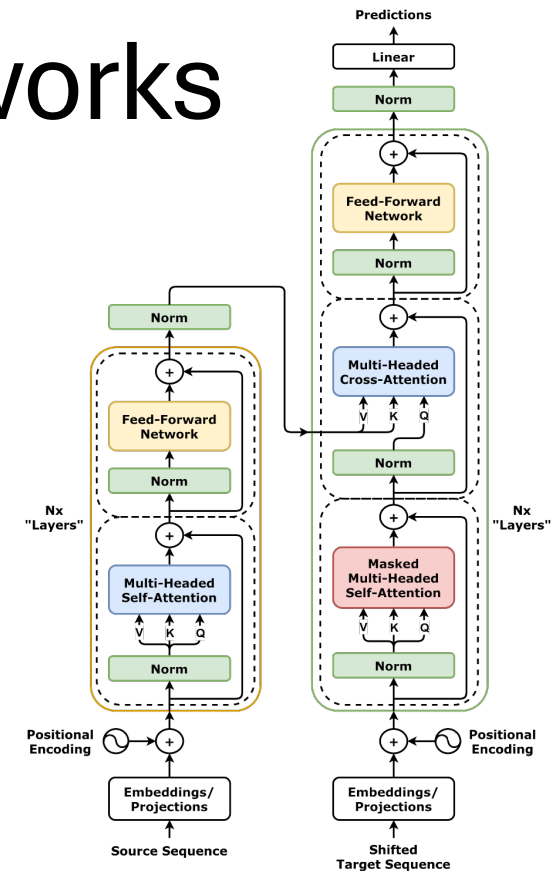
Developed by researchers at [Google](#) and is based on the multi head [attention](#) mechanism



Llion Jones
Studied BSc & MSc @ UoB

https://scholar.google.com/citations?user=3_P5VwAAAAJ&hl=en

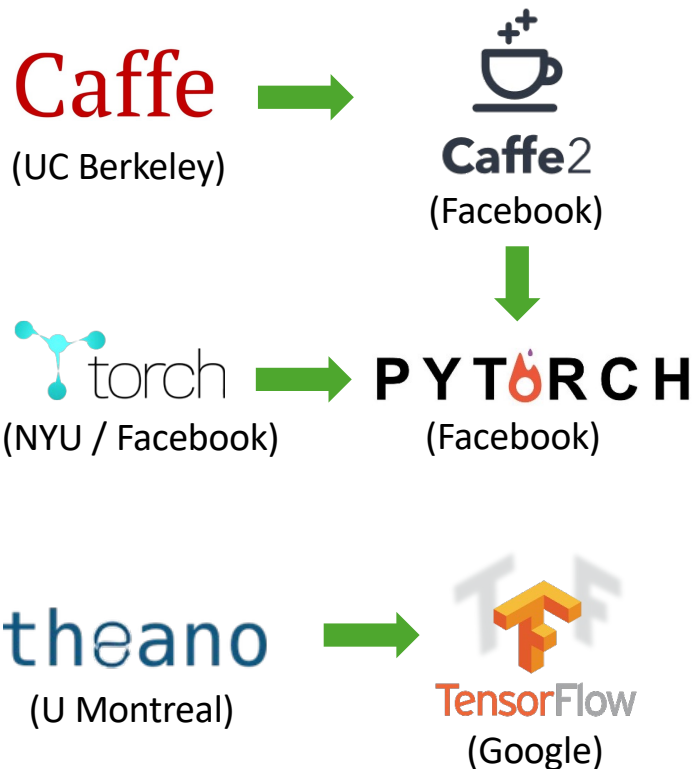
A standard Transformer architecture, showing on the left an encoder, and on the right a decoder.



Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin, [Attention Is All You Need](#), NIPS 2017

Implementation

Deep learning framework



 **PaddlePaddle**
(Baidu)

 **Chainer**

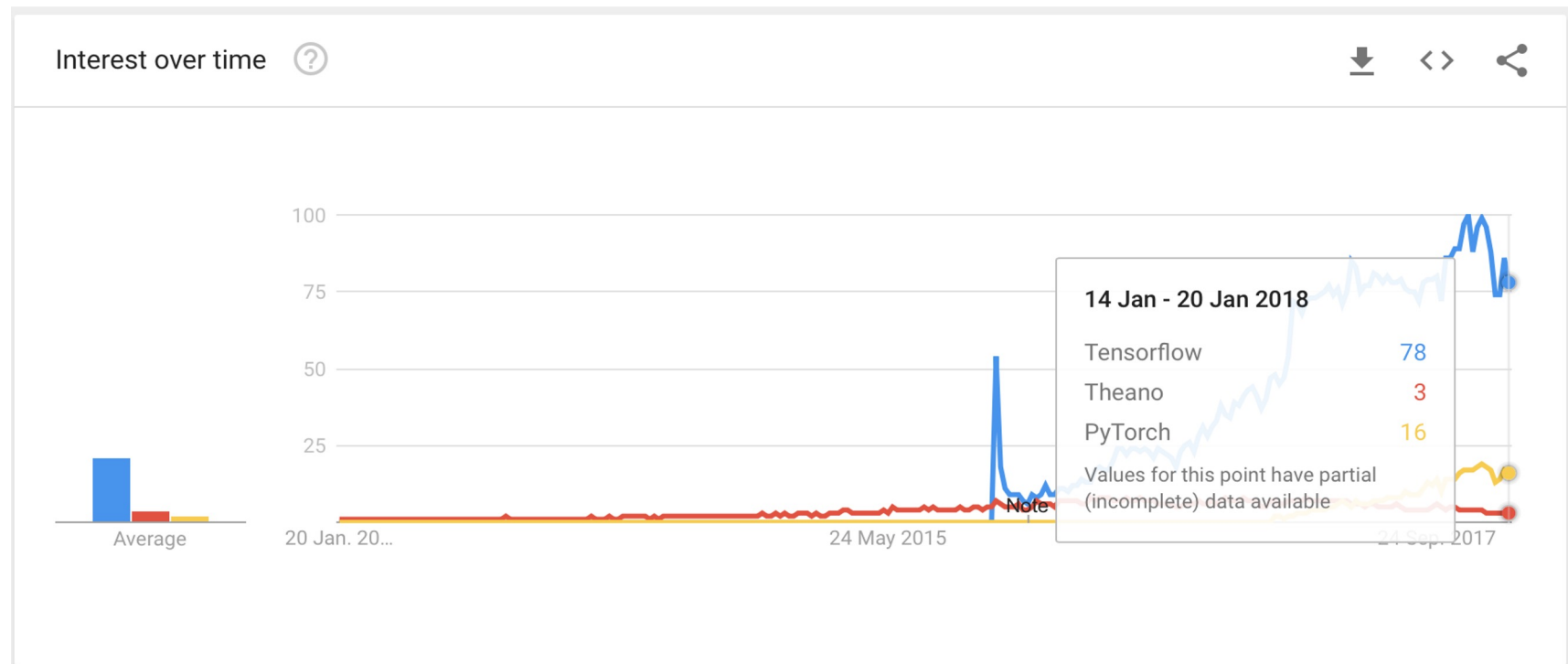
dmlc
mxnet
(Amazon)

 Microsoft
CNTK
(Microsoft)

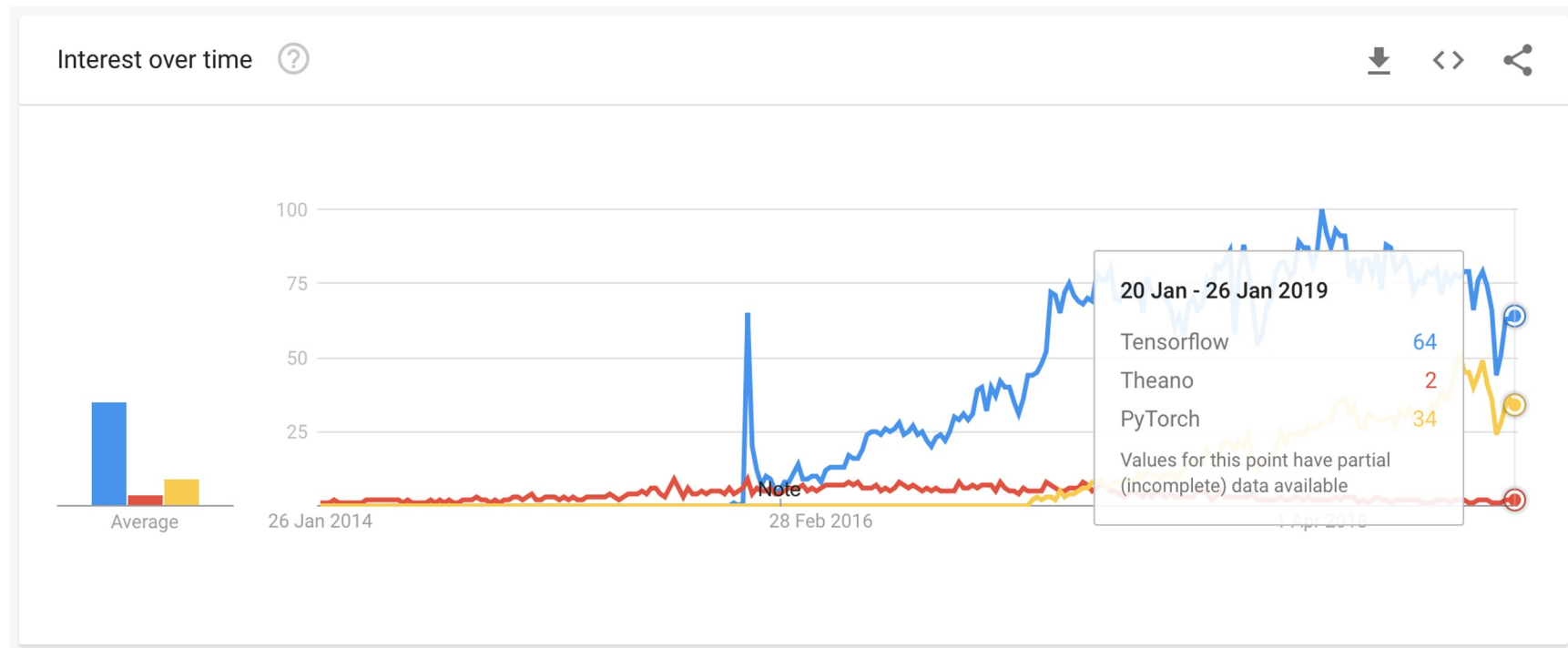
 **AlphaGo**
(Google)

 **DL4J**
+ **MatConvNet** etc...

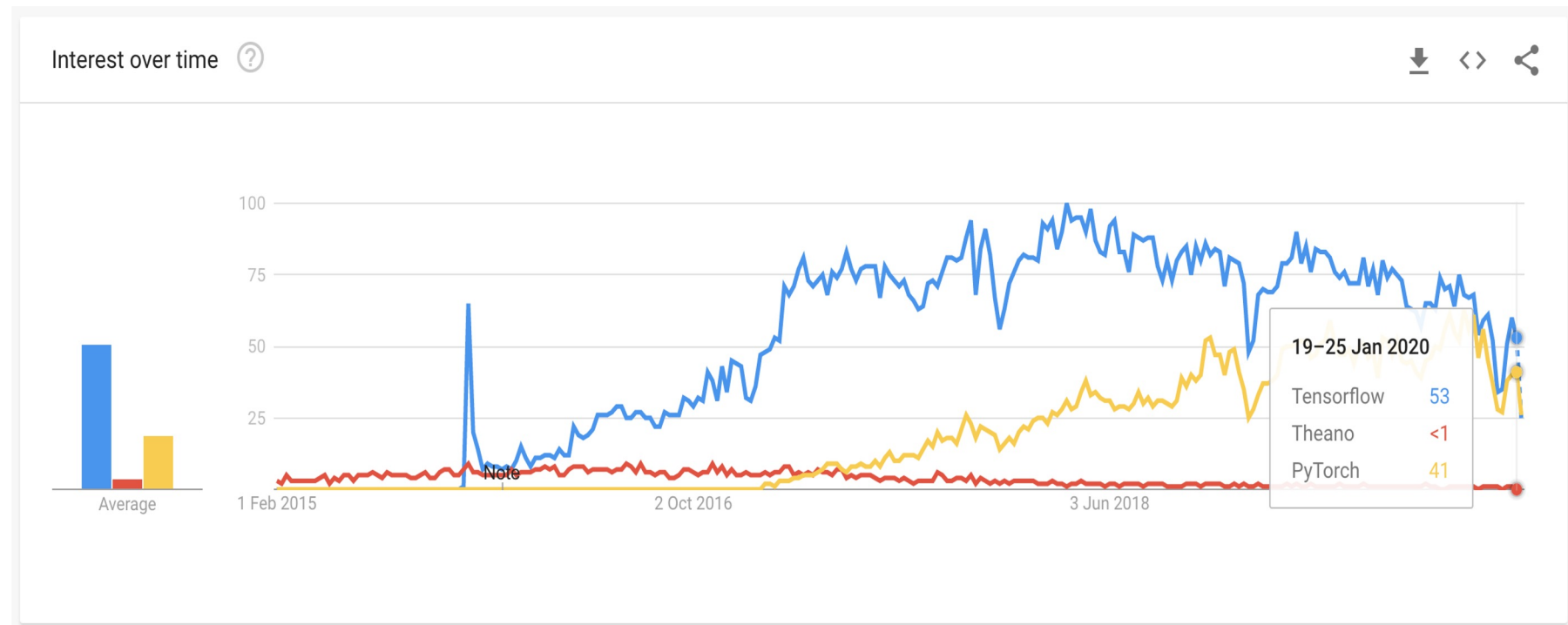
Implementation



Implementation



Implementation



Implementation

Interest over time ?



Implementation

Aim of these frameworks

- Easily build big computational graphs
- Easily compute gradients in computational graphs
- Run it all efficiently on GPU (wrap cuDNN, cuBLAS, etc)

Implementation

Suggestions?

- ✓ For beginners – **Keras**
- ✓ For Google Cloud Platform – **Tensorflow**
- ✓ For research – **PyTorch**
- ✓ For Amazon Web Services (AWS) – **MXNET**
- ✓ For MS Azura – **CNTK**
- ✓ For Java developers – **DL4J**
- ✓ For IOS developers – **CoreML**
- ✓ For Interoperability - **ONNX**

Implementation



Implementation

Static vs Dynamic: Graphs

TensorFlow 1.X: Build graph once, then run many times (**static**)

```
N, D, H = 64, 1000, 100
x = tf.placeholder(tf.float32, shape=(N, D))
y = tf.placeholder(tf.float32, shape=(N, D))
w1 = tf.Variable(tf.random_normal((D, H)))
w2 = tf.Variable(tf.random_normal((H, D)))

h = tf.maximum(tf.matmul(x, w1), 0)
y_pred = tf.matmul(h, w2)
diff = y_pred - y
loss = tf.reduce_mean(tf.reduce_sum(diff ** 2, axis=1))
grad_w1, grad_w2 = tf.gradients(loss, [w1, w2])

learning_rate = 1e-5
new_w1 = w1.assign(w1 - learning_rate * grad_w1)
new_w2 = w2.assign(w2 - learning_rate * grad_w2)
updates = tf.group(new_w1, new_w2)

with tf.Session() as sess:
    sess.run(tf.global_variables_initializer())
    values = {x: np.random.randn(N, D),
              y: np.random.randn(N, D),}
    losses = []
    for t in range(50):
        loss_val, _ = sess.run([loss, updates],
                               feed_dict=values)
```

Build
graph

Run each
iteration

PyTorch: Each forward pass defines a new graph (**dynamic**)

```
import torch
from torch.autograd import Variable

N, D_in, H, D_out = 64, 1000, 100, 10
x = Variable(torch.randn(N, D_in), requires_grad=False)
y = Variable(torch.randn(N, D_out), requires_grad=False)
w1 = Variable(torch.randn(D_in, H), requires_grad=True)
w2 = Variable(torch.randn(H, D_out), requires_grad=True)

learning_rate = 1e-6
for t in range(500):
    y_pred = x.mm(w1).clamp(min=0).mm(w2)
    loss = (y_pred - y).pow(2).sum()

    if w1.grad: w1.grad.data.zero_()
    if w2.grad: w2.grad.data.zero_()
    loss.backward()

    w1.data -= learning_rate * w1.grad.data
    w2.data -= learning_rate * w2.grad.data
```

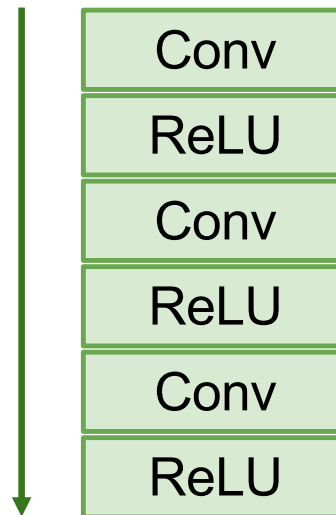
New graph each iteration

Implementation

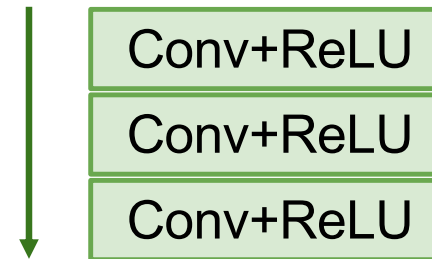
Static vs Dynamic: optimisation

With static graphs, framework can **optimize** the graph for you before it runs!

The graph you wrote



Equivalent graph with **fused operations**



Implementation

Static vs Dynamic: Serialisation

Static

Once graph is built, can **serialize** it and run it without the code that built the graph!

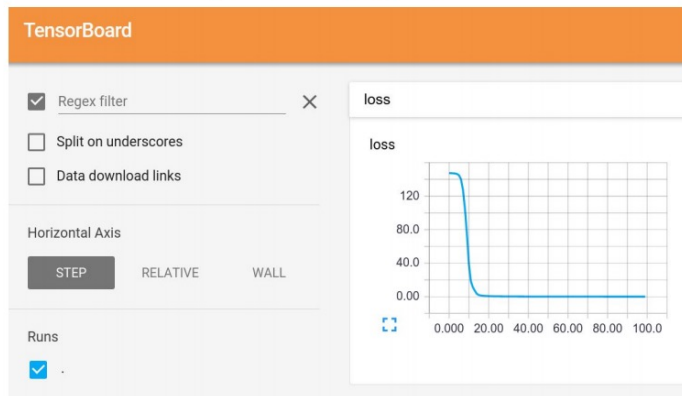
Dynamic

Graph building and execution are intertwined, so always need to keep code around

Implementation

TensorFlow: Tensorboard

Add logging to code to record loss, stats, etc
Run server and get pretty graphs!



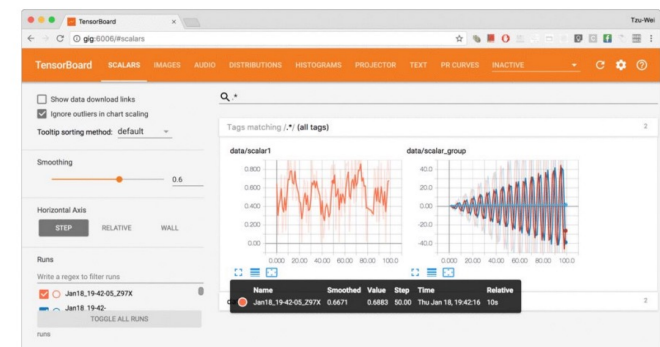
Visualisation

PyTorch: tensorboardX

A python wrapper around
Tensorflow's web-based
visualization tool.

`pip install tensorboardx`

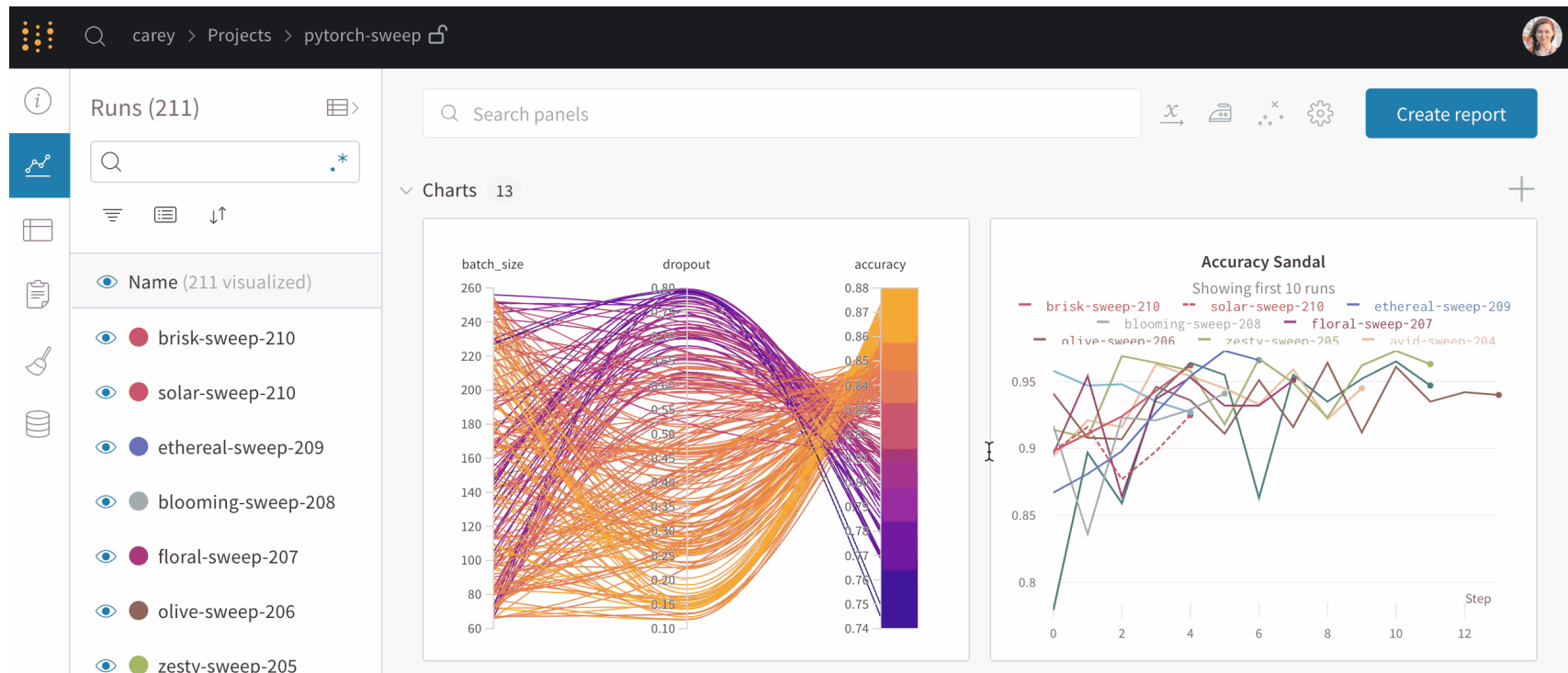
<https://github.com/lanpa/tensorboardX>



This image is licensed under CC-BY 4.0; no changes were made to the image

Implementation

Visualisation

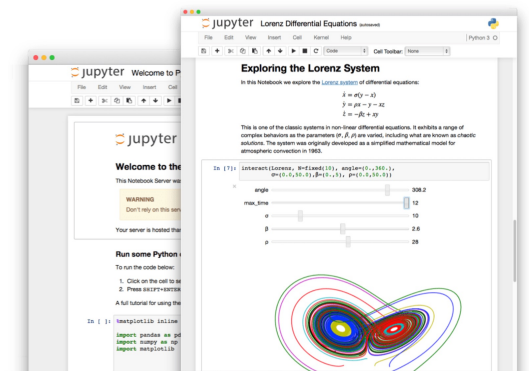


<https://wandb.ai/site>

Implementation

Editors

- Jupyter [Notebook](#) (formerly IPython Notebooks) is a [web-based interactive](#) computational environment for creating Jupyter notebook documents.
- The Jupyter Notebook has become a popular user interface for [Cloud computing](#)
 - E.g. [Amazon's SageMaker](#) Notebooks, [Google's Colaboratory](#), [Microsoft's Azure Notebook](#), and Jupyo.

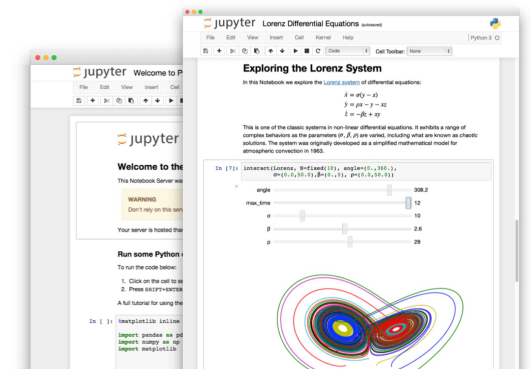


Implementation

Editors



- ✓ Jupyter Notebooks really shine when you are still in the **prototyping phase**.
- ✓ The code is written in **independent cells**, which are executed individually. This allows the user to test a specific block of code in a project without having to execute the code from the start of the script.
- ✓ Incredibly flexible, interactive and many powerful extensions.
- ✓ Allow you to run other languages besides Python, like R, SQL, etc.



Implementation

Editors

- **PyCharm** is one of the most widely used IDEs for Python programming language.
 - PyCharm integrates its tools and libraries such as NumPy and Matplotlib, allowing you work with array viewers and interactive plots.
 - Interesting features such as a code editor, errors highlighting, a powerful debugger with a graphical interface, besides of Git integration, SVN, and Mercurial.



Implementation

Inside a computer

CPU or GPU?



Slide credit: Fei-Fei Li & Justin Johnson & Serena Yeung

Implementation

CPU or GPU?

Spot the CPU!
(central processing unit)



This image is licensed under CC-BY 2.0



Slide credit: Fei-Fei Li & Justin Johnson & Serena Yeung

Implementation

CPU or GPU?

Spot the GPUs!
(graphics processing unit)



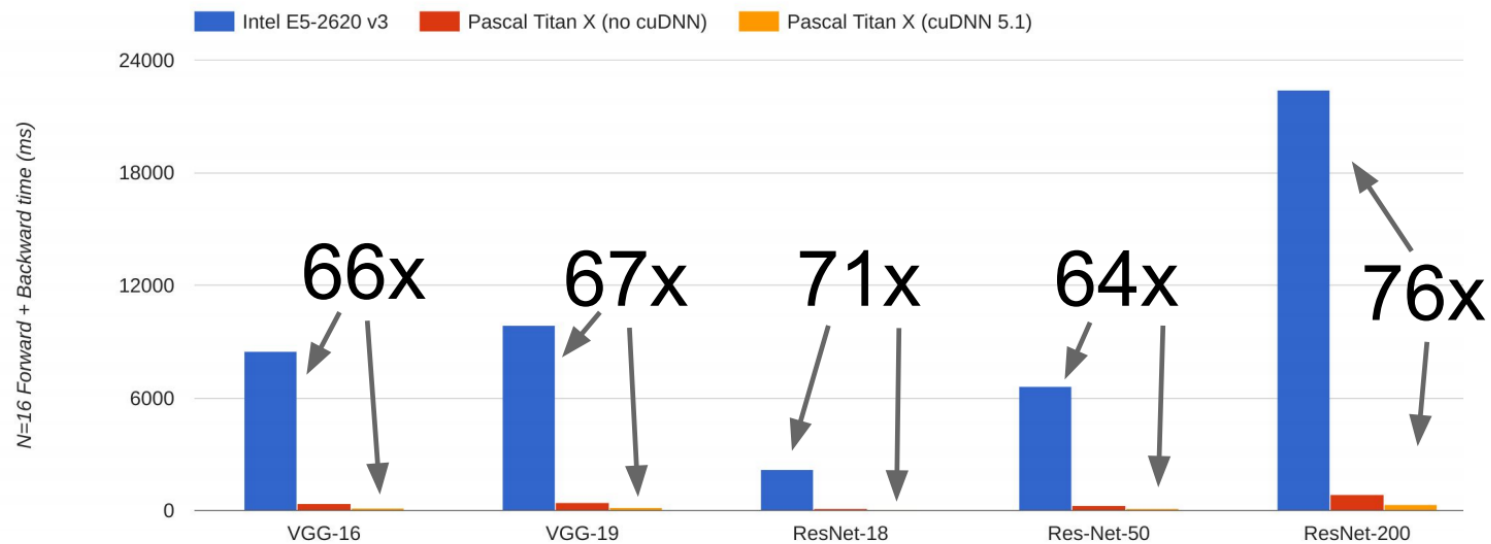
[This image](#) is in the public domain



Slide credit: Fei-Fei Li & Justin Johnson & Serena Yeung

Implementation

CPU or GPU?



Data from <https://github.com/jcjohnson/cnn-benchmarks>

Slide credit: Fei-Fei Li & Justin Johnson & Serena Yeung

Free online courses

- **Summer School**

- Deep Learning & Reinforcement Learning Summer School
 - (2017) <https://goo.gl/4WthXN>
 - (2018) <https://goo.gl/Z7stFe>
- Machine Learning Summer School (MPI) <https://goo.gl/5hQtA1>

- **Module**

- DeepLearning.ai (Andrew Ng) <https://goo.gl/mMr4AW>
- CS231n (Stanford) <https://goo.gl/WTLZkg>
- CS224d (Stanford) <https://goo.gl/nmY6Ws>
- Fast.ai (Fast.ai) <https://goo.gl/aBkesx>

- **Reinforcement Learning**

- Deep RL bootcamp (UC Berkeley) <https://goo.gl/i6CbtR>
- UCL lectures (DeepMind) <https://goo.gl/gF7EoY>
- CS294 (UC Berkeley) <https://goo.gl/d17a5x>

Free deep learning books

1. [Deep Learning](#) by *Ian Goodfellow, Yoshua Bengio and Aaron Courville*
2. [Deep Learning Tutorial](#) by *LISA Lab, University of Montreal*
3. [Deep Learning: Methods and Applications](#) by *Li Deng and Dong Yu*
4. [First Contact with TensorFlow, get started with Deep Learning Programming](#) by *Jordi Torres*
5. [Neural Networks and Deep Learning](#) by *Michael Nielsen*
6. [A Brief Introduction to Neural Networks](#) by *David Kriesel*
7. [Neural Network Design \(2nd edition\)](#) by *Martin T. Hagan, Howard B. Demuth, Mark H. Beale and Orlando D. Jess*
8. [Neural Networks and Learning Machines \(3rd edition\)](#) by *Simon Haykin*

Q & A